AI Fog Computing — Demonstration Guide

This guide describes the **implemented** system. All values are sourced directly from the codebase.

---

1. System Architecture

The platform has three distinct tiers:

Tier	Technology	Role
Browser SPA	React 18 + TypeScript + Vite	UI, simulation engine (runs entirely client-side)
API server	Express + Mongoose (Node.js)	Dataset import, persistence, fog-node planning
Database	MongoDB (local)	Stores imported dataset records

Key point: The scheduling simulation — task generation, DRL scoring, node-state updates, metrics — all runs in the browser. The Express backend is only involved during dataset import (`/api/datasets/import`) and providing the node-plan contract endpoint (`GET /api/nodes/plan?records=N`). Vite proxies every `/api` request to `http://localhost:4000` during development.

Mermaid diagram

```
graph LR
    Browser["Browser (React SPA)\nSimulation engine runs here"] -->|"/api/* (Vite proxy)"| Express["Express API\n:4000"]
    Express --> MongoDB["MongoDB\n:27017"]
    Express -->|"planFogNodes()"| Browser
    MongoDB -->|"stored records count"| Express
```

---

2. Load Balancing and the Four Scheduling Strategies

Source: `src/utils/scheduler.ts`

2.1 DRL Optimized (`drl_optimized`)

A simplified Deep Reinforcement Learning inspired scoring system. For every active fog node that has enough free CPU and memory to accept the task, a composite score is computed:

```
score = (resourceScore * 0.3
       + distanceScore * 0.2
       + energyScore   * 0.2
       + loadScore     * 0.3)
       * priorityMultiplier
```

Component definitions:

Component	Formula	Meaning
`resourceScore`	`((maxCpu - currentCpu)/maxCpu + (maxMemory - currentMemory)/maxMemory) / 2`	Fraction of CPU and memory still free
`distanceScore`	`1 / (1 + distance/100)`	Closer nodes score higher
`energyScore`	`1 / (1 + energyConsumption/100)`	Lower current energy wins
`loadScore`	`1 - (processingTasks.length / 10)`	Fewer concurrent tasks wins

Priority multipliers:

Priority	Multiplier
`low`	1
`medium`	1.5
`high`	2
`critical`	3

Cloud fallback: After scoring all fog nodes the scheduler also computes a cloud score via `calculateCloudScore(task)`:

```
cloudScore = 0.7 * priorityPenalty
priorityPenalty = { low: 0.8, medium: 0.6, high: 0.4, critical: 0.1 }
```

If `cloudScore > bestFogScore`, the task is offloaded to the cloud node. This means high-priority tasks are strongly biased towards fog, while low-priority tasks can land on cloud when fog nodes are loaded.

2.2 Priority Aware (`priority_aware`)

Routes `critical` and `high` priority tasks to the fog node with the smallest `distance` value (lowest latency proxy). All other priorities go to the first available fog node, or cloud if none can accept the task.

2.3 Load Balanced (`load_balanced`)

Selects the fog node with the fewest tasks currently in `processingTasks`. Falls back to cloud when no fog node has capacity.

2.4 Energy Efficient (`energy_efficient`)

Selects the fog node with the lowest `energyConsumption`. Falls back to cloud when no fog node has capacity.

---

3. Fog-Node Creation

Sources: `src/utils/fogNodes.ts`, `server/src/lib/fogNodes.js` (identical logic in both).

3.1 The clamp rule

```
rawCount = ceil(recordCount / 1000)
nodeCount = clamp(rawCount, 3, 12)          // never fewer than 3, never more than 12
capacityMult = rawCount > 12 ? ceil(rawCount / 12) : 1
```

Constants: `NODE_PER_RECORDS = 1000`, `MIN_NODES = 3`, `MAX_NODES = 12`.

3.2 Example: 10,000 records

```
rawCount = ceil(10000 / 1000) = 10
nodeCount = clamp(10, 3, 12) = 10           // 10 fog nodes created
capacityMult = 1                             // 10 <= 12, no scaling needed
```

3.3 Node profiles (assigned round-robin by index)

Profile	maxCpu	maxMemory
`small` (index % 3 == 0)	80	150 MB
`medium` (index % 3 == 1)	100	200 MB
`large` (index % 3 == 2)	120	250 MB

Node distances: `distance = 10 + index * 5` (Fog Node 1 = 10, Fog Node 2 = 15, …).

3.4 Beyond-cap multiplier

When `recordCount` implies more than 12 nodes (e.g., 15,000 records → rawCount 15 > 12), the system still creates 12 nodes but scales each node's capacity:

```
capacityMult = ceil(rawCount / 12)
// e.g. rawCount=15 → mult=2 → small node gets maxCpu=160, maxMemory=300
```

---

4. Performance Metrics

Source: `src/hooks/useSimulation.ts` (`calculateMetrics`, the `useEffect` on `[tasks, fogNodes, cloudNode]`), `src/components/MetricsDashboard.tsx`.

Metric	Unit	Definition
Total Tasks	count	Total tasks in the `tasks` array (all statuses)
Completion Rate	%	`(completedTasks / totalTasks) * 100`
Avg Latency	ms	Mean `actualLatency` of completed tasks (falls back to `expectedLatency` while tasks are still processing)
Energy Used	kW	Sum of `energyConsumption` across all fog nodes (live, per tick)
Fog Utilization	%	`(sum of currentCpu across nodes / sum of maxCpu across nodes) * 100`
Cloud Offload	%	`(tasks assigned to cloud / totalTasks) * 100`

Per-node energy model (computed in `updateNodeStates`):

```
node.energyConsumption = currentCpu * 0.5 + currentMemory * 0.1
```

where `currentCpu` is the sum of CPU requirements of all tasks currently processing on that node, and `currentMemory` is the equivalent sum for memory.

The loopRef fix: React's `setInterval` captures the callback once at `startSimulation` time. Without a fix, subsequent state changes (new tasks, new fog nodes) would be invisible to the ticking loop — causing frozen metrics and stale node status. The fix:

```
const loopRef = useRef(simulationLoop);
useEffect(() => { loopRef.current = simulationLoop; }, [simulationLoop]);

// interval always calls through the ref, not a stale closure:
intervalRef.current = setInterval(() => loopRef.current(), 1000);
```

This ensures every 1-second tick invokes the latest `simulationLoop` with current state.

---

5. End-to-End Workflow

Step 1 — Login

Navigate to `http://localhost:5173`. The admin login gate (added in the platform upgrade) prompts for credentials. Default: **admin / admin123** (set in `server/.env`). Invalid credentials are rejected.

Step 2 — Import dataset

In the Dataset section, upload a CSV file (or use the pre-loaded SECOM data). The file is `POST`ed to `/api/datasets/import`. The backend pipeline:

1. Parse CSV rows.
2. **Clean and validate** each row (`server/src/lib/cleaning.js`): verify `device_type` is one of `healthcare | traffic | industrial` ; `priority` is `low | medium | high | critical` ; `cpu` in (0, 100]; `memory` in (0, 8192]; `latency` in (0, 100000].
3. **Deduplicate** by `rowHash = sha1(deviceType|priority|cpu|memory|latency|sensorId)` . Rows with identical hashes are dropped.
4. Persist valid, unique rows to MongoDB.
5. Return `{ valid, invalidCount, duplicatesRemoved, recordCount }` .

Important note on the SECOM dataset: The UCI SECOM dataset contains 1,566 rows but has extremely low field diversity — only two priority values and no `sensor_id` column. After cleaning and deduplication, this source yields only approximately **16 unique records** rather than 1,566. This is a property of the dataset, not a bug. For a richer simulation, use a synthetic CSV or a dataset with varied `device_type` , `priority` , and numeric fields.

Step 3 — Automatic node plan

After import the frontend builds fog nodes entirely **client-side**: `applyRecordCount(count)` calls `buildFogNodes(count)` (in `src/utils/fogNodes.ts`) to reshape the topology. The network topology view updates immediately. The backend exposes `GET /api/nodes/plan?records=N` as an API/contract endpoint for external consumers and parity verification — the browser UI does **not** call that route.

Step 4 — Select scheduling strategy

Choose one of: DRL Optimized, Priority Aware, Load Balanced, Energy Efficient. The `AIScheduler` is re-instantiated with the new strategy.

Step 5 — Start simulation

Click **Start**. The simulation loop fires immediately and then every 1 second:

1. `generateTasks()` — injects batches of up to 10 dataset tasks per tick (or generates random tasks from IoT devices if dataset mode is off).
2. `scheduleTasks()` — assigns each `pending` task to a node using the active strategy.
3. `processTasks()` — randomly completes or fails `processing` tasks (40 % completion chance in dataset mode, 5 % failure chance).
4. `updateNodeStates()` — recomputes `currentCpu` , `currentMemory` , `energyConsumption` for each node.
5. `calculateMetrics()` — updates the six dashboard metrics.

Step 6 — Observe

- **Network Topology** — live node graph showing fog nodes, the cloud node, and active task connections.
- **Metrics Dashboard** — six cards (only cards with non-zero values are shown).
- **Task Table** — per-task rows with `assignedTo`, `actualLatency`, `energyConsumed`, `completedAt`.
- **Performance Chart** — rolling 50-point history of latency, energy, and utilization.

---

6. Key Concepts

Fog vs Cloud

Property	Fog nodes	Cloud node
Latency	Low (distance 10–65 ms proxy)	Fixed 150 ms
Capacity	Limited (clamped 3–12 nodes)	Unlimited
Energy	Tracked and modelled	Not modelled
Preference	High-priority tasks	Low-priority overflow

Priority multipliers

The DRL scoring formula multiplies the composite score by the task's priority multiplier: `low×1`, `medium×1.5`, `high×2`, `critical×3`. A critical task's score can be up to 3× that of a low-priority task on the same node, making it far more likely to win the best available fog slot.

Energy model

Node energy consumption is computed each tick as:

```
energyConsumption = Σ(cpuRequirement_i * 0.5) + Σ(memoryRequirement_i * 0.1)
```

where the sums are over all tasks currently processing on that node. The dashboard reports the sum of all node consumptions in kW units.

Deduplication by rowHash

Before any record is stored, the server computes:

```
rowHash = sha1("deviceType|priority|cpu|memory|latency|sensorId")
```

A `Set` of seen hashes is maintained during import. Any row whose hash was already seen is counted as a duplicate and skipped. This is a pure in-memory deduplication pass; the database `rowHash` field can

additionally enforce uniqueness at the storage layer.